

Working with Data

Session 2 — What happens when tables meet?

Block 4 — Data that is not a table yet: APIs and JSON

This block in one sentence

An API's answer is a document, not a table. Find the grain hiding inside it, write it down as rows — and read which way its numbers go.

What we cover

- What an API is: a URL that returns data. Status codes, and why 200 is not "correct"
- Being polite: one request, a timeout, a name, a cache
- A real answer from the World Bank: the metadata, the records, **a quarter of the truth**
- JSON shapes and the tables they hide: one table per grain
- DuckDB reads JSON; when a six-line Python loop is clearer
- A rate has a direction; the rate of a whole is a weighted mean
- A share of a total: the total as a query inside the query
- Git, AI, then Lab 4 and Homework 2

What an API is: a URL that returns data

```
https://api.worldbank.org/v2/country/all/indicator/SP.P0P.T0TL  
?format=json&date=2010:2024&per_page=1000&page=1
```

- **Where:** `api.worldbank.org/v2` · **what:** population (`SP.P0P.T0TL`) for every country
- **How much, and which part:** after `?`, the **parameters** — format, years, page size, **page number**
- Three steps, always: **ask** (with a timeout) → **check** the status code → **read** the body

Most code you will read uses the `requests` library: `requests.get(url, timeout=10)`, then `.status_code`, then `.json()`. The starter uses Python's own `urllib`: the same three steps, nothing to install.

Status codes — and why 200 is not "correct"

Code	Means	What to do
200	the server answered	<i>now</i> check the answer
404	no such thing	read your URL
429	too many requests	slow down; you are the problem
500	the server broke	try later

Real answers, asked on 19 September 2026:

- World Bank, **page 9 of 4**: status **200**, and an empty list
- World Bank, an **indicator that does not exist**: status **200**, and an error message *inside* the data
- Frankfurter, an unknown currency: status 404

Being polite to a public server

- **One request, not sixty.** A room that all asks at once is 60 requests for one answer.
- **A timeout**, always: `timeout=10` . Never wait forever.
- **Say who you are.** Frankfurter answers Python's built-in `urllib` , under its default name, with `403 Forbidden` ; with a `User-Agent` header that names your program, `200` .
- **Cache it.** Save the answer to `data/raw/api/` , write down the URL and the date, and read the file from then on.
- **No keys in code.** Every API in this course needs none. When one does, the key lives outside the repo.

One request, live — watch

```

cell 1 import urllib.request

url = ("https://api.worldbank.org/v2/country/all/indicator/SP.POP.TOTL"
      "?format=json&date=2010:2024&per_page=1000&page=1")
request = urllib.request.Request(url, headers={"User-Agent": "ecbs5294-lecture"})  # say who is asking
try:
    with urllib.request.urlopen(request, timeout=10) as response:                # never wait forever
        print("status:", response.status)
        live = json.load(response)
        print(live[0])
except OSError as problem:              # no Wi-Fi, a timeout, a refusal: the cached page below is the same answer
    print("No live answer:", problem, "-> use the cached page")

```

```

status: 200
{'page': 1, 'pages': 4, 'per_page': 1000, 'total': 3975, 'sourceid': '2',
'lastupdated': '2026-07-13'}

```

captured 25 September 2026

Predict, then watch: open the saved answer

lecture.ipynb , section 2:

```
page1 = json.load(open("data/raw/api/worldbank/SP.POP.TOTL_page1.json"))
print(type(page1), len(page1))
print(page1[0])
print(len(page1[1]), "records on this page")
```

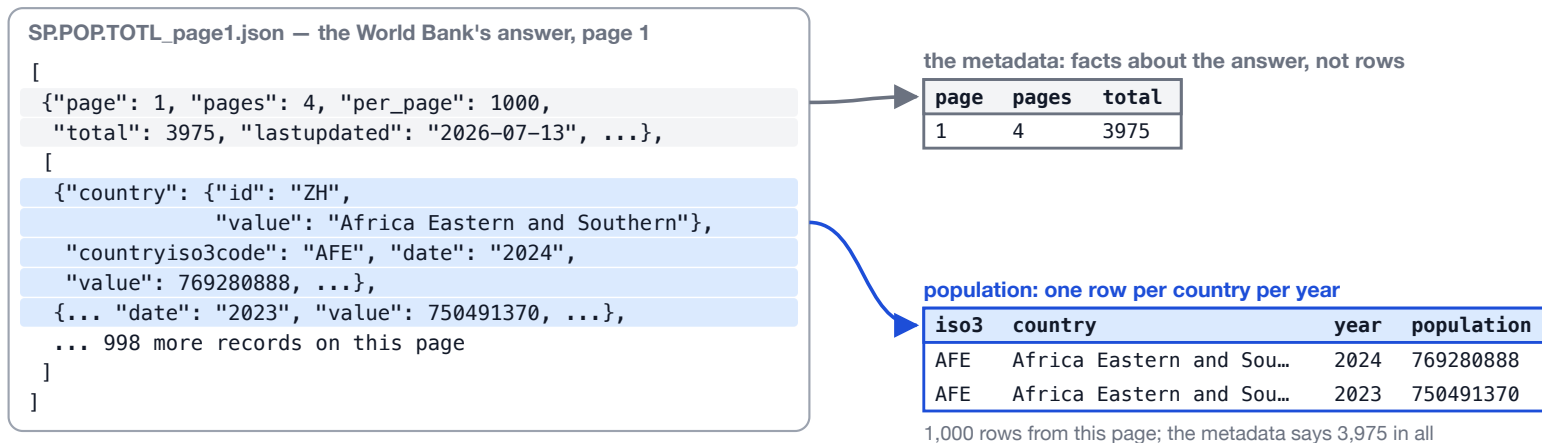
Predict: what kind of thing is `page1` , and how long is it? What will its first element be? Write one reason.

Open the saved answer — what it printed

```
cell 2  page1 = json.load(open("data/raw/api/worldbank/SP.POP.TOTL_page1.json"))
        print(type(page1), len(page1))
        print(page1[0])
        print(len(page1[1]), "records on this page")
```

```
<class 'list'> 2
{'page': 1, 'pages': 4, 'per_page': 1000, 'total': 3975, 'sourceid': '2',
 'lastupdated': '2026-07-13'}
1000 records on this page
```

Watch: what does DuckDB see?



Section 3: `read_json_auto` on the same file, and the JSON type of each row DuckDB made.

Watch: what DuckDB made of it

```
cell 3 con.sql("""  
    SELECT json_type(json) AS element  
    FROM read_json_auto('data/raw/api/worldbank/SP.P0P.T0TL_page1.json')  
    """).df()
```

	element
0	OBJECT
1	ARRAY

Predict, then watch: the list inside the document becomes rows

Section 4:

```
SELECT record->>'countryiso3code'      AS iso3,
       record->'country'->>'value'      AS country,
       CAST(record->>'date' AS INTEGER) AS year,
       CAST(record->>'value' AS BIGINT) AS population
FROM (SELECT unnest(CAST(json AS JSON[])) AS record           -- one row per element
      FROM read_json_auto('.../SP.P0P.TOTL_page1.json')
      WHERE json_type(json) = 'ARRAY')                       -- skip the metadata
```

unnest : a list becomes one row per element. **->** goes one level in; **->>** takes the value out as text.

Predict: is Hungary in this table? Write one reason.

The list inside the document becomes rows — what it printed

```
cell 4 con.sql("""
CREATE OR REPLACE TABLE population_page1 AS
SELECT record->>'countryiso3code'      AS iso3,
       record->'country'->>'value'      AS country,
       CAST(record->>'date' AS INTEGER) AS year,
       CAST(record->>'value' AS BIGINT) AS population
FROM (SELECT unnest(CAST(json AS JSON[])) AS record           -- one row per element of the list
      FROM read_json_auto('data/raw/api/worldbank/SP.POP.TOTL_page1.json')
      WHERE json_type(json) = 'ARRAY')                       -- element 0 is the metadata: skip it
""")
con.sql("""
SELECT COUNT(*) AS n_rows, COUNT(DISTINCT country) AS n_countries,
       COUNT(*) FILTER (WHERE country = 'Hungary') AS hungary_rows
FROM population_page1
""").df()
```

	n_rows	n_countries	hungary_rows
0	1000	67	0

Predict, then watch: every page, counted against **total**

Section 5:

```
records = []
for page in range(1, page1[0]["pages"] + 1):
    doc = json.load(open(f"data/raw/api/worldbank/SP.POP.TOTL_page{page}.json"))
    records += doc[1]
print(len(records), "records; the metadata says", page1[0]["total"])
```

Predict: what would it print **without** the **+ 1** ? Write one reason.

Every page, counted against **total** — what it printed

```
cell 5 records = []
for page in range(1, page1[0]["pages"] + 1):
    doc = json.load(open(f"data/raw/api/worldbank/SP.POP.TOTL_page{page}.json"))
    records += doc[1]
print(len(records), "records; the metadata says", page1[0]["total"])
```

3975 records; the metadata says 3975

```
cell 5b records_short = []
for page in range(1, page1[0]["pages"]):          # no + 1: range stops before its end
    doc = json.load(open(f"data/raw/api/worldbank/SP.POP.TOTL_page{page}.json"))
    records_short += doc[1]
print(len(records_short), "records without the + 1; the metadata says", page1[0]["total"])
```

3000 records without the + 1; the metadata says 3975

Watch: all four pages in one query

Section 6: the same query with a `*` in the path — `.../SP.POP.TOTL_page*.json` — into a table called `population`. Section 11 comes back to this table.

```
cell 6 con.sql("""
    CREATE OR REPLACE TABLE population AS
    SELECT record->'countryiso3code'      AS iso3,
           record->'country'->'value'     AS country,
           CAST(record->'date' AS INTEGER) AS year,
           CAST(record->'value' AS BIGINT) AS population
    FROM (SELECT unnest(CAST(json AS JSON[])) AS record
          FROM read_json_auto('data/raw/api/worldbank/SP.POP.TOTL_page*.json')
          WHERE json_type(json) = 'ARRAY')
    """)
print(con.sql("SELECT COUNT(*), COUNT(DISTINCT country) FROM population").fetchone())
con.sql("SELECT * FROM population WHERE country = 'Hungary' AND year = 2024").df()
```

(3975, 265)

	iso3	country	year	population
0	HUN	Hungary	2024	9562065

JSON shapes, and the table each one hides

A list of records → rows

document

```
[{"iso3": "HUN", "year": 2024, "pop": 9562065},
 {"iso3": "HUN", "year": 2023, "pop": 9592186}]
```

iso3	year	pop
HUN	2024	9562065
HUN	2023	9592186

A dictionary keyed by id → the key becomes a column

document

```
{"HUN": {"name": "Hungary", "capital": "Budapest"},
 "BRA": {"name": "Brazil", "capital": "Brasilia"}}
```

id	name	capital
HUN	Hungary	Budapest
BRA	Brazil	Brasilia

A dictionary of dictionaries → (outer key, inner key, value)

document

```
{"2017-01-02": {"BRL": 3.4265},
 "2017-01-03": {"BRL": 3.3916}, ...}
```

date	currency	value
2017-01-02	BRL	3.4265
2017-01-03	BRL	3.3916

A list inside a record → a child table, keyed back

document

```
{"order_id": "04a2b4ee...", "state": "SP",
 "items": [{"n": 1, "price": 145.00},
 {"n": 2, "price": 19.99}]}
```

orders

order_id	state
04a2b4ee...	SP

order_items

order_id	n	price
04a2b4ee...	1	145.00
04a2b4ee...	2	19.99

key (order_id, n); order_id points back

One table per grain — even in the World Bank's answer

- Every population record repeats its country: `{"id": "HU", "value": "Hungary"}`, 15 times, once per year
- That is a second grain hiding in the first: **the country**, stored once, in the country file (`country_page1.json` : 295 entries, of which 78 say `region: "Aggregates"`)
- So: `population (iso3, year, value)` at country-year grain, `countries (iso3, name, region)` at country grain, joined on `iso3`

This morning's joins are how the two grains meet again.

Predict, then watch: a dictionary of dictionaries

`toy_rates.json` is **three real days** copied from an ECB rates answer, nothing changed:

```
{"amount": 1.0, "base": "EUR",  
  "rates": {"2017-01-02": {"BRL": 3.4265},  
            "2017-01-03": {"BRL": 3.3916},  
            "2017-01-04": {"BRL": 3.3784}}}
```

Section 7 prints the file, then `DESCRIBE SELECT * FROM read_json_auto('data/raw/api/toy_rates.json')`.

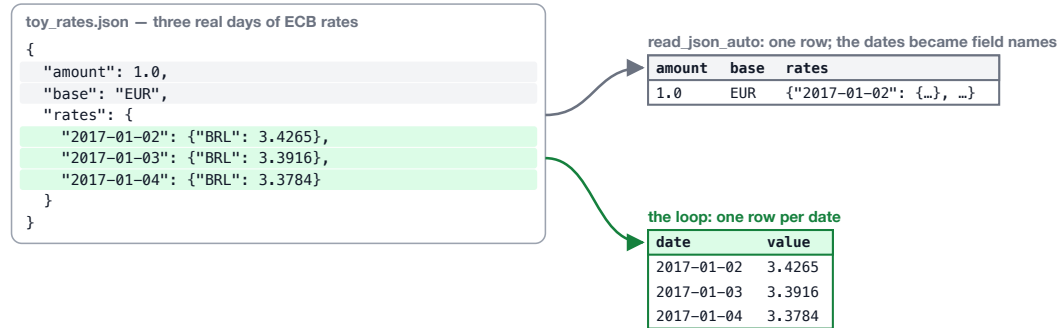
Predict: how many rows does DuckDB make of it, and what type is `rates` ? Write one reason.

A dictionary of dictionaries — what it printed

cell 7b `con.sql("DESCRIBE SELECT * FROM read_json_auto('data/raw/api/toy_rates.json')").df()`

	column_name	column_type	null	key	default	extra
0	amount	DOUBLE	YES	None	None	None
1	base	VARCHAR	YES	None	None	None
2	rates	STRUCT("2017-01-02" STRUCT(BRL DOUBLE), "2017-...	YES	None	None	None

Watch: struct access



Section 8:

```
con.sql("""
    SELECT base, rates."2017-01-02".BRL AS brl
    FROM read_json_auto('data/raw/api/toy_rates.json')
    """).df()
```

To ask for a day, you have to **write the day as a name**. For two years of days, that is 555 names.

Watch: struct access — what it printed

cell 8

```
con.sql("""  
    SELECT base, rates."2017-01-02".BRL AS brl  
    FROM read_json_auto('data/raw/api/toy_rates.json')  
""").df()
```

	base	brl
0	EUR	3.4265

Watch: six lines turn the keys into rows

Section 9:

```
cell 9 toy = json.load(open("data/raw/api/toy_rates.json"))
rows = []
for day, values in toy["rates"].items():
    rows.append((day, values["BRL"]))
con.execute("CREATE OR REPLACE TABLE toy_rates (date DATE, value DOUBLE)")
con.executemany("INSERT INTO toy_rates VALUES (?, ?)", rows)
con.sql("SELECT * FROM toy_rates").df()
```

	date	value
0	2017-01-02	3.4265
1	2017-01-03	3.3916
2	2017-01-04	3.3784

`.items()` hands you each key **with** what is under it: the key becomes a value in a row.

Which way does the rate go?

Predict, then watch. The document says `"amount": 1.0, "base": "EUR"`. On 2 January 2017, `"BRL": 3.4265`.

If the base is EUR, what is `3.4265` ?

- **A.** reais for one euro
- **B.** euros for one real

Then, on paper: 100 reais on that day are how many euros? Write one reason.

Which way does the rate go? — what it printed

```
cell 10 toy = json.load(open("data/raw/api/toy_rates.json"))
print("base:", toy["base"], "| amount:", toy["amount"])
for day, values in toy["rates"].items():
    print(day, values["BRL"], "->", f"{100 / values['BRL']:.2f}", "euros for 100 reais")
```

```
base: EUR | amount: 1.0
2017-01-02 3.4265 -> 29.18 euros for 100 reais
2017-01-03 3.3916 -> 29.48 euros for 100 reais
2017-01-04 3.3784 -> 29.60 euros for 100 reais
```

Predict, then watch: the rate of a whole is a weighted mean

Section 11, on section 6's `population`. Brazil is about twenty times Hungary's size.

```
WITH g AS (
    SELECT country,
           MAX(CASE WHEN year = 2010 THEN population END) AS pop_2010,
           MAX(CASE WHEN year = 2024 THEN population END) AS pop_2024
    FROM population
    WHERE country IN ('Brazil', 'Hungary')
    GROUP BY country
)
SELECT SUM(pop_2024) / SUM(pop_2010) AS together,           -- each country at its own growth
       AVG(pop_2024 / pop_2010)    AS plain_average,        -- the two growths, averaged
       MIN(pop_2024 / pop_2010)    AS lowest,
       MAX(pop_2024 / pop_2010)    AS highest
FROM g
```

Predict: is `together` equal to `plain_average`? If not, nearer Brazil's growth or Hungary's? Would "between `lowest` and `highest`" tell them apart? One reason.

The rate of a whole is a weighted mean — what it printed

```
cell 11 together, plain, lowest, highest = con.sql("""
    WITH g AS (
        SELECT country,
               MAX(CASE WHEN year = 2010 THEN population END) AS pop_2010,
               MAX(CASE WHEN year = 2024 THEN population END) AS pop_2024
        FROM population
        WHERE country IN ('Brazil', 'Hungary')
        GROUP BY country
    )
    SELECT SUM(pop_2024) / SUM(pop_2010) AS together,      -- each country at its own growth
           AVG(pop_2024 / pop_2010)    AS plain_average,  -- the two growths, averaged
           MIN(pop_2024 / pop_2010)    AS lowest,
           MAX(pop_2024 / pop_2010)    AS highest
    FROM g
""").fetchone()
print(f"together {together:.4f} | plain average {plain:.4f} | lowest {lowest:.4f} | highest {highest:.4f}")
```

```
together 1.0877 | plain average 1.0253 | lowest 0.9562 | highest 1.0945
```

Predict, then watch: each country's share of the whole

Section 12a, on the same rows. A share is a part over its whole:

```
con.sql("""
    SELECT country,
           population / SUM(population) AS share
    FROM population
    WHERE year = 2010 AND country IN ('Brazil', 'Hungary')
""").df()
```

Predict: two shares, one per country, or an error? One reason.

Each country's share — what it printed

cell 12a

```
try:
    con.sql("""
        SELECT country,
               population / SUM(population) AS share
        FROM population
        WHERE year = 2010 AND country IN ('Brazil', 'Hungary')
        """).df()
except duckdb.Error as e:
    print("DuckDB refused:", str(e).splitlines()[0])
```

DuckDB refused: Binder Error: column "country" must appear in the GROUP BY clause or must be part of an aggregate function.

The total as a query inside the query

cell 12b

```
con.sql("""
    SELECT country, population,
           population / (SELECT SUM(population) FROM population
                        WHERE year = 2010 AND country IN ('Brazil', 'Hungary')) AS share
    FROM population
    WHERE year = 2010 AND country IN ('Brazil', 'Hungary')
""").df()
```

	country	population	share
0	Brazil	193701929	0.950909
1	Hungary	10000023	0.049091

The lab's file — described, not opened

`data/raw/api/frankfurter_eur.json` has the toy's shape, grown:

- **two years** of ECB business days, 2016-09-01 to 2018-10-31
- **two currencies** inside every day: `rates → date → currency → value`
- `amount` and `base` beside `rates`, as in the toy

Fetches once by `scripts/fetch.py`; `DATA.md` records the URL, the date, and its SHA-256.

Offline-first: the cache is the input

- The analysis reads `data/raw/api/...`, never the network. It runs on a plane, and next year.
- `uv run python scripts/fetch.py` shows the request and checks the cache; `--live` asks once and compares. It **never** overwrites the cache.
- Asking again is a decision: the answer may have changed. Refresh on purpose, re-run every check, say so in the note.

Git thread: raw is committed, made is ignored

- `data/raw/api/` is **committed**, with `DATA.md` : it is the evidence
- A table your notebook **makes** — the rates table, as a CSV in `data/silver/` — is **ignored**: run the notebook and it comes back
- Session 2's habit, again: a commit that changes a join carries **the three counts** in its message

AI thread: the assistant reads the field names

Ask it: *"Convert my BRL revenue to EUR using these cached rates."* It will write a join.

Then ask: **"Which way does the rate go? How do you know?"**

Then check the document's `base` yourself.

The assistant reasons from the field names. The document's own metadata is the source of truth.

The symptom class

No error. A number in the right column, the right kind of number, and wrong —

- a rate read the wrong way round, or
- rows that found no partner and quietly left

Neither shows up in a range check. **Only a line you compute yourself does.**

5-minute stand-and-stretch — start the Lab 4 clone first

In your terminal (Git Bash on Windows, Terminal on macOS), in the folder where you keep course work:

```
git clone https://github.com/earino/ecbs5294-lab04-json-api.git  
cd ecbs5294-lab04-json-api  
uv sync
```

Start it, then **stand up**. It downloads while you stretch. Back in five.

Homework 2 assigned

Many tables and an API: the board-deck numbers — 15% of the grade, about **6–7 hours**. Due **Saturday 17 October, 23:59**, on Moodle.

- A colleague's KPI notebook for a board deck, **half done**: four KPIs drafted, their totals disagree with the headline; two not started. `BRIEF.md` defines all six.
- Evidence **before** each repair; every repair **a query; three counts** for every join
- A **reconciliation table**: one identity per KPI — a sum, a ratio's two parts, or a weighted mean — and an **assertions** cell
- `DIAGNOSIS.md`, a short **Results** note for the board, commits, the archive, a 60–90 s video

Lab 4 — The revenue that was converted the wrong way

Minute 1: check the clone and `uv sync` finished; open the **folder** in VS Code; open **`notebooks/report.ipynb`**; kernel → `.venv`. Run all.

- The report converts Olist's revenue into euros. **The euro total is larger than the reais total**, though one real was worth less than one euro on every day of the period. Find out why.
- **Disclosed, not to diagnose:** 625 orders fall on days with no ECB rate (weekends, ECB holidays). Your monthly query must keep them; **your check must count them**.
- Read the document **before** its numbers. Then: the rates table the right way round (B, a loop); **one order converted by hand** (C — the proof that matters); monthly euros (D); the check (E).
- **By minute 22, start `DIAGNOSIS.md`** : five minutes, in class. At minute 33, your neighbour.

The last ten minutes — explain it to a neighbour

Speaker, about a minute, pointing at the screen, not reading the note: symptom → cause → evidence → change → verification.

Listener asks:

1. Show me the query that proves it.
2. Why was it wrong — not just where?
3. **Suppose the cache ended on 2018-08-31.** What happens to September's revenue in your join?

Swap. Unsure, or you disagree? **Hand up.** Then the answer, for everyone. Before you leave:

DIAGNOSIS.md to the Lab 4 checkpoint on Moodle.